

AWS EC2 Lab Guide

AWS EC2 Lab Guide - VM with Startup Script Web Server

Lab Overview

1. Launch an EC2 instance using AWS Console
2. Configure a startup script to automatically deploy a web server
3. Set up security groups for web access
4. Access the webpage via public IP/DNS

Prerequisites

- AWS Account with EC2 permissions
- Basic understanding of HTML and Linux commands

Part 1: AWS Console (GUI) Approach

Step 1: Launch EC2 Instance

1. **Navigate to EC2 Service**
 - Login to AWS Console
 - Search for "EC2" and click on the service
 - Click "Launch Instance"
2. **Configure Instance Details**

Name and Tags:

- Name: `student-web-server-demo`

Application and OS Images:

- Select "Amazon Linux 2023" (free tier eligible)
- Architecture: 64-bit (x86)

Instance Type:

- Select `t2.micro` (free tier eligible)

Key Pair:

- Create new key pair or select existing one
- Name: `student-demo-keypair`
- Key pair type: RSA
- Private key file format: `.pem`
- Click "Create key pair" and download the `.pem` file

Step 2: Configure Network Settings

1. **Security Group Configuration**
 - Click "Edit" next to Network settings

- Security group name: web-server-sg
- Description: Security group for web server demo

2. Inbound Rules (Add these rules):

- **Rule 1: SSH**
 - Type: SSH
 - Protocol: TCP
 - Port: 22
 - Source: 0.0.0.0/0 (or your IP for better security)
- **Rule 2: HTTP**
 - Type: HTTP
 - Protocol: TCP
 - Port: 80
 - Source: 0.0.0.0/0
- **Rule 3: Custom TCP (for our demo app)**
 - Type: Custom TCP
 - Protocol: TCP
 - Port: 8080
 - Source: 0.0.0.0/0

Step 3: Configure Storage

- Keep default settings (8 GB gp3 root volume)

Step 4: Advanced Details - User Data Script

1. Expand "Advanced Details" section
2. Scroll down to "User data" text box
3. Paste the following startup script:

```
#!/bin/bash
# Update system
yum update -y

# Install required packages
yum install -y httpd git nodejs npm

# Start and enable Apache
systemctl start httpd
systemctl enable httpd

# Create a simple HTML webpage
cat > /var/www/html/index.html << 'EOF'
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>AWS EC2 Demo - Student Lab</title>
```

```

<style>
  body {
    font-family: 'Arial', sans-serif;
    margin: 0;
    padding: 0;
    background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
    min-height: 100vh;
    display: flex;
    align-items: center;
    justify-content: center;
  }
  .container {
    background: white;
    padding: 2rem;
    border-radius: 15px;
    box-shadow: 0 10px 30px rgba(0,0,0,0.3);
    text-align: center;
    max-width: 600px;
    margin: 20px;
  }
  h1 {
    color: #333;
    margin-bottom: 1rem;
  }
  .info-card {
    background: #f8f9fa;
    padding: 1rem;
    border-radius: 8px;
    margin: 1rem 0;
    border-left: 4px solid #667eea;
  }
  .timestamp {
    color: #666;
    font-size: 0.9em;
  }
  .server-info {
    background: #e9ecef;
    padding: 0.5rem;
    border-radius: 4px;
    font-family: monospace;
    margin: 0.5rem 0;
  }
  .success {
    color: #28a745;
    font-weight: bold;
  }
</style>
</head>
<body>
  <div class="container">
    <h1>🚀 AWS EC2 Demo Server</h1>
    <div class="success">✅ Startup Script Executed Successfully!</div>
  </div>

```

```

    <div class="info-card">
      <h3>Server Information</h3>
      <div class="server-info">Instance ID: <span id="instance-
id">Loading ... </span></div>
      <div class="server-info">Public IP: <span id="public-ip">Loading ...
</span></div>
      <div class="server-info">Region: <span id="region">Loading ... </span>
</div>
    </div>

    <div class="info-card">
      <h3>Lab Objectives Completed</h3>
      <p>✔ EC2 Instance launched successfully</p>
      <p>✔ User data script executed</p>
      <p>✔ Web server installed and running</p>
      <p>✔ Security groups configured</p>
      <p>✔ Public access enabled</p>
    </div>

    <div class="timestamp">
      Page loaded: <span id="current-time"></span>
    </div>

    <div style="margin-top: 2rem;">
      <p><strong>Student Name:</strong> _____</p>
      <p><strong>Lab Date:</strong> _____</p>
    </div>
  </div>

  <script>
    // Get current time
    document.getElementById('current-time').textContent = new
Date().toLocaleString();

    // Fetch AWS metadata (these will work when running on EC2)
    fetch('http://169.254.169.254/latest/meta-data/instance-id')
      .then(response => response.text())
      .then(data => document.getElementById('instance-id').textContent =
data)
      .catch(() => document.getElementById('instance-id').textContent = 'Not
available');

    fetch('http://169.254.169.254/latest/meta-data/public-ipv4')
      .then(response => response.text())
      .then(data => document.getElementById('public-ip').textContent = data)
      .catch(() => document.getElementById('public-ip').textContent = 'Not
available');

    fetch('http://169.254.169.254/latest/meta-data/placement/region')
      .then(response => response.text())
      .then(data => document.getElementById('region').textContent = data)

```

```

        .catch(() => document.getElementById('region').textContent = 'Not
available');
    </script>
</body>
</html>
EOF

# Set proper permissions
chown apache:apache /var/www/html/index.html
chmod 644 /var/www/html/index.html

# Create a simple Node.js app on port 8080
mkdir -p /home/ec2-user/nodeapp
cd /home/ec2-user/nodeapp

cat > package.json << 'EOF'
{
  "name": "ec2-demo-app",
  "version": "1.0.0",
  "description": "Simple demo app for EC2 lab",
  "main": "server.js",
  "scripts": {
    "start": "node server.js"
  }
}
EOF

cat > server.js << 'EOF'
const http = require('http');
const fs = require('fs');

const server = http.createServer((req, res) => {
  res.writeHead(200, {'Content-Type': 'text/html'});
  res.end(`
<!DOCTYPE html>
<html>
<head>
  <title>Node.js Demo on EC2</title>
  <style>
    body { font-family: Arial; margin: 40px; background: #f0f0f0; }
    .container { background: white; padding: 20px; border-radius: 8px; }
    h1 { color: #2196F3; }
  </style>
</head>
<body>
  <div class="container">
    <h1>🟢 Node.js Application Running on EC2</h1>
    <p><strong>Port:</strong> 8080</p>
    <p><strong>Status:</strong> ☒ Running</p>
    <p><strong>Time:</strong> ${new Date().toISOString()}</p>
    <p><strong>Process ID:</strong> ${process.pid}</p>
    <hr>

```

```

        <p>This demonstrates that both Apache (port 80) and Node.js (port
        8080) can run simultaneously on the same EC2 instance.</p>
    </div>
</body>
</html>
`);
});

server.listen(8080, () => {
    console.log('Node.js server running on port 8080');
});
EOF

# Change ownership and start Node.js app
chown -R ec2-user:ec2-user /home/ec2-user/nodeapp
cd /home/ec2-user/nodeapp

# Start Node.js app in background
su - ec2-user -c "cd /home/ec2-user/nodeapp && nohup node server.js > app.log 2>&1
&"

# Create a status file to confirm script completion
echo "User data script completed at $(date)" > /tmp/userdata-status.txt

# Log completion
echo "EC2 Demo setup completed successfully" >> /var/log/user-data.log

```

Step 5: Launch Instance

1. Review Configuration

- Review all settings in the summary
- Ensure security groups allow HTTP traffic

2. Launch Instance

- Click "Launch Instance"
- Wait for instance to reach "running" state (2-3 minutes)

Step 6: Test the Web Server

1. Get Instance Information

- Go to EC2 Dashboard → Instances
- Select your instance
- Copy the "Public IPv4 address" and "Public IPv4 DNS"

2. Access the Websites

- **Apache Server (Port 80):**
 - Open browser and go to: `http://YOUR-PUBLIC-IP`
 - Or use: `http://YOUR-PUBLIC-DNS`
- **Node.js App (Port 8080):**
 - Open browser and go to: `http://YOUR-PUBLIC-IP:8080`

- Or use: `http://YOUR-PUBLIC-DNS:8080`

3. Verify Functionality

- Both pages should load successfully
- The main page should show instance metadata
- Screenshots recommended for lab documentation

Part 2: Troubleshooting and Verification

Check User Data Script Execution

1. Connect to Instance via SSH

```
chmod 400 student-demo-keypair.pem
ssh -i student-demo-keypair.pem ec2-user@YOUR-PUBLIC-IP
```

2. Verify Services are Running

```
# Check Apache status
sudo systemctl status httpd

# Check if Node.js is running
ps aux | grep node

# Check user data script completion
cat /tmp/userdata-status.txt

# View user data logs
sudo cat /var/log/cloud-init-output.log
```

3. Manual Service Start (if needed)

```
# Start Apache if not running
sudo systemctl start httpd

# Start Node.js app manually
cd /home/ec2-user/nodeapp
nohup node server.js > app.log 2>&1 &
```

Common Issues and Solutions

Issue	Symptom	Solution
Page not loading	Connection timeout	Check security groups allow HTTP (port 80)
Node.js not accessible	Port 8080 timeout	Verify security group has port 8080 open
Services not started	503/404 errors	SSH in and manually start services
Metadata not showing	"Not available" text	Normal for external access to metadata

Part 3: Lab Exercises for Students

Exercise 1: Basic Verification

- ☐ Launch EC2 instance with provided user data script
- ☐ Verify both web servers are accessible
- ☐ Take screenshots of both pages
- ☐ Document public IP and DNS names

Exercise 2: Customization

- ☐ SSH into the instance
- ☐ Modify the HTML content in `/var/www/html/index.html`
- ☐ Add your name and student ID to the page
- ☐ Refresh browser to see changes

Exercise 3: Log Analysis

- ☐ Check cloud-init logs: `sudo cat /var/log/cloud-init-output.log`
- ☐ View Apache access logs: `sudo tail -f /var/log/httpd/access_log`
- ☐ Monitor Node.js app logs: `tail -f /home/ec2-user/nodeapp/app.log`

Exercise 4: Security Group Modification

- ☐ Temporarily remove HTTP rule from security group
- ☐ Try accessing the website (should fail)
- ☐ Add the rule back and verify access restored

Part 4: Advanced Customization

Custom User Data Script Template

```
#!/bin/bash
# Student can customize this section
STUDENT_NAME="Your Name Here"
COURSE_CODE="AWS-101"

# Update and install packages
yum update -y
yum install -y httpd

# Create personalized webpage
cat > /var/www/html/index.html << EOF
<!DOCTYPE html>
<html>
<head>
  <title>${STUDENT_NAME} AWS Lab</title>
  <style>body{font-family:Arial;text-align:center;margin-top:50px;}</style>
</head>
<body>
  <h1>Hello from ${STUDENT_NAME}</h1>
  <h2>Course: ${COURSE_CODE}</h2>
```



```
<p>Instance launched at: $(date)</p>
<p>Public IP: $(curl -s http://169.254.169.254/latest/meta-data/public-ipv4)
</p>
</body>
</html>
EOF

# Start web server
systemctl start httpd
systemctl enable httpd
```

Part 5: Clean Up

Stop/Terminate Instance

1. **Stop Instance** (preserves data, stops billing for compute):
 - Select instance → Instance State → Stop instance
2. **Terminate Instance** (deletes everything):
 - Select instance → Instance State → Terminate instance
 - ⚠ **Warning:** This permanently deletes the instance

Clean Up Resources

- Delete security groups (if custom created)
- Delete key pairs (if no longer needed)
- Remove any elastic IPs (if created)

Part 6: Success Criteria

Students should be able to demonstrate:

- ✅ Successfully launched EC2 instance
- ✅ User data script executed without errors
- ✅ Apache web server accessible on port 80
- ✅ Node.js application accessible on port 8080
- ✅ Security groups properly configured
- ✅ Instance metadata displayed correctly
- ✅ Can SSH into instance for troubleshooting

Quick Reference URLs

After launching your instance, you'll access:

- **Main Website:** `http://YOUR-PUBLIC-IP/`
- **Node.js App:** `http://YOUR-PUBLIC-IP:8080/`
- **SSH Access:** `ssh -i keypair.pem ec2-user@YOUR-PUBLIC-IP`

Estimated Time

- **Setup:** 10-15 minutes

- **Testing:** 5 minutes
- **Exercises:** 15-20 minutes
- **Total:** ~30-40 minutes